



Université de
Technologie de
Compiègne

LO17/AI31 : Indexation et Recherche d'Information

Rapport de Projet DM

Membre du binôme 1 :
Clothilde MARKO-LAFON (50%)

Membre du binôme 2 :
Caio ARTUS (50%)

Groupe :
LO17 - Groupe TD 1



Sommaire

1	Préparation du corpus	3
1.1	Présentation des fichiers	3
1.2	Extraction des champs à partir du HTML	3
1.3	Construction du fichier XML et de l'index	4
2	Anti-dictionnaire	5
2.1	Choix de l'unité documentaire	5
2.2	Calcul du TF-IDF	5
2.3	Détermination de l'anti-dictionnaire	5
2.4	Génération du corpus sans stop word	5
3	Lemmatisation et indexation du corpus	6
3.1	Lemmatisation du corpus filtré	6
3.2	Affinage de l'anti-dictionnaire	6
3.3	Construction des fichiers inverses	7
3.4	La classe DataCleaner	7
4	Correcteur orthographique	8
4.1	Pipeline de correction	8
4.2	Génération des candidats par recherche par préfixe	8
4.3	Départage par distance de Levenshtein	8
4.4	Vulnérabilité de l'approche et alternative	8
5	Traitement des requêtes	9
5.1	Format d'une requête	9
5.2	Extraction des dates	9
5.3	Extraction des mots clés	9
5.4	Classe Pretraiteur	10
6	Moteur de recherche et évaluation	11
6.1	Chargement de l'index	11
6.2	Recherche	11
6.3	Classement des résultats	11
6.4	Interface en ligne de commande et web	11
6.5	Évaluation du moteur de recherche	12



Introduction

Dans le cadre du cours LO17 sur l'indexation et la recherche de données, nous avons pour projet de réaliser un moteur de recherche sur un corpus de documents de l'ADIT. Ce travail englobe toutes les étapes : des pages web brutes HTML qui composent le corpus au moteur de recherche abouti. Le résultat final est *ADITSearch*, un moteur de recherche qui peut traiter des requêtes en langage naturel et retourner des documents pertinents via une recherche booléenne. *ADITSearch* atteint un score F1 de 0.79 sur un ensemble de 10 requêtes complexes, ce qui montre une réelle capacité à chercher de l'information. Il est implémenté en Python avec une approche orientée objet lui permettant d'être extensible par de futurs travaux.

Ce rapport détaille toutes les étapes de ce travail, en mettant l'accent sur les choix techniques, ainsi que sur les avantages et inconvénients des méthodes utilisées.



Partie 1

Préparation du corpus

L'objectif de ce premier TD est de construire un fichier XML unique et structuré à partir des fichiers HTML bruts de l'archive ADIT, afin de disposer d'une base propre et homogène pour toutes les étapes d'indexation qui suivront.

1.1 Présentation des fichiers

L'archive est constituée de bulletins électroniques publiés entre 2011 et 2014. Chaque bulletin est un fichier HTML contenant un ou plusieurs articles, répartis dans des rubriques telles que *Focus*, *Événement*, *Actualité Innovation* ou *En direct des laboratoires*. On souhaite en extraire les champs suivants : le numéro de l'article et du bulletin, la date de parution, la rubrique, le titre, le texte, l'auteur, les images avec leur URL et leur légende, et les informations de contact.

1.2 Extraction des champs à partir du HTML

1.2.1 Architecture du parseur

On structure le parseur autour de deux classes aux responsabilités distinctes. `BulletinParser` prend en charge un unique fichier HTML : il l'ouvre, en extrait tous les champs, et produit le fragment XML correspondant. `CorpusParser` orchestre l'ensemble : il itère sur tous les fichiers du répertoire, instancie un `BulletinParser` par fichier, puis assemble les fragments pour produire le fichier `corpus.xml`.

1.2.2 Double parsing du HTML

On commence par passer chaque fichier dans `BeautifulSoup` pour normaliser le HTML potentiellement malformé, puis on transmet le résultat à `etree.HTML()` de `lxml` pour obtenir un DOM interrogeable en XPath. Cette combinaison tire parti de la robustesse de `BeautifulSoup` pour la correction du HTML et de l'expressivité de XPath pour l'extraction des champs.

1.2.3 Extraction par chemins XPath

Tous les champs sont ciblés par des chemins XPath identifiés en inspectant la structure HTML depuis le navigateur. On applique `strip()` sur les valeurs extraites pour nettoyer les espaces parasites, et `replace()` pour isoler certaines informations encodées dans le texte, comme le numéro de bulletin toujours précédé de la chaîne "BE France".

Extraction du texte. Le texte de l'article partage la même cellule HTML que les images. Pour n'extraire que le contenu éditorial, on utilise un prédicat XPath qui exclut les blocs contenant des images (`not(ancestor::div[img])`) et les spans de légende (`style88`).

Extraction des images. Les images sont ciblées via `div[img]`, l'URL récupérée via l'attribut `src`, et la légende via `following-sibling::span`.

Extraction de l'auteur. Le bloc auteur suit la forme Organisation - Nom - Email : adresse. Une expression régulière souple avec `re.IGNORECASE` en extrait les trois sous-champs. En cas d'échec, les champs sont mis à `None` plutôt que de lever une exception, afin de ne pas interrompre le traitement des autres articles.

Contact. La structure du bloc contact étant trop hétérogène, on l'inclut tel quel dans la balise `<contact>` du XML.



1.3 Construction du fichier XML et de l'index

1.3.1 Formation du fichier XML

Chaque `BulletinParser` produit un fragment XML que `CorpusParser` accroche à la racine `<corpus>`. Chaque article est encapsulé dans une balise `<document>`, conformément à la structure imposée par le TD. L'ensemble est consolidé dans le fichier `clean_corpus.xml`.

1.3.2 Tokenisation

La tokenisation est assurée par `CorpusTokenizer`. On segmente uniquement sur les espaces et les apostrophes, ce qui **conserve le tiret** à l'intérieur des tokens (`bien-être` reste un token unique, préservant ainsi les termes composés). Chaque token est mis en minuscules, et tous les caractères spéciaux sont supprimés à l'exception de `\w` et du tiret. Les chiffres sont conservés pour permettre la détection des entités numériques au TD4. Enfin, le titre et le texte sont **concaténés avant tokenisation**, de sorte que le titre contribue à la fréquence globale des tokens pour le calcul du TF-IDF.

1.3.3 Construction de l'index inversé

Les fichiers inverses sont construits par la classe `Index` depuis le script `make_index.py`. La méthode `build()` distingue les **champs tokenisés** (titre, texte), découpés par `CorpusTokenizer` et indexés avec leur fréquence et la liste des documents associés, des **champs bruts** (rubrique, bulletin, date, auteur, contact), mis en minuscules et indexés tels quels pour permettre une correspondance exacte. La présence d'images est indexée via le token "image" dès qu'une balise `<images>` est détectée. Les résultats sont sauvegardés en TSV, un fichier par section, ainsi qu'un fichier `lemmes_corpus.csv` consolidant l'ensemble des tokens uniques.

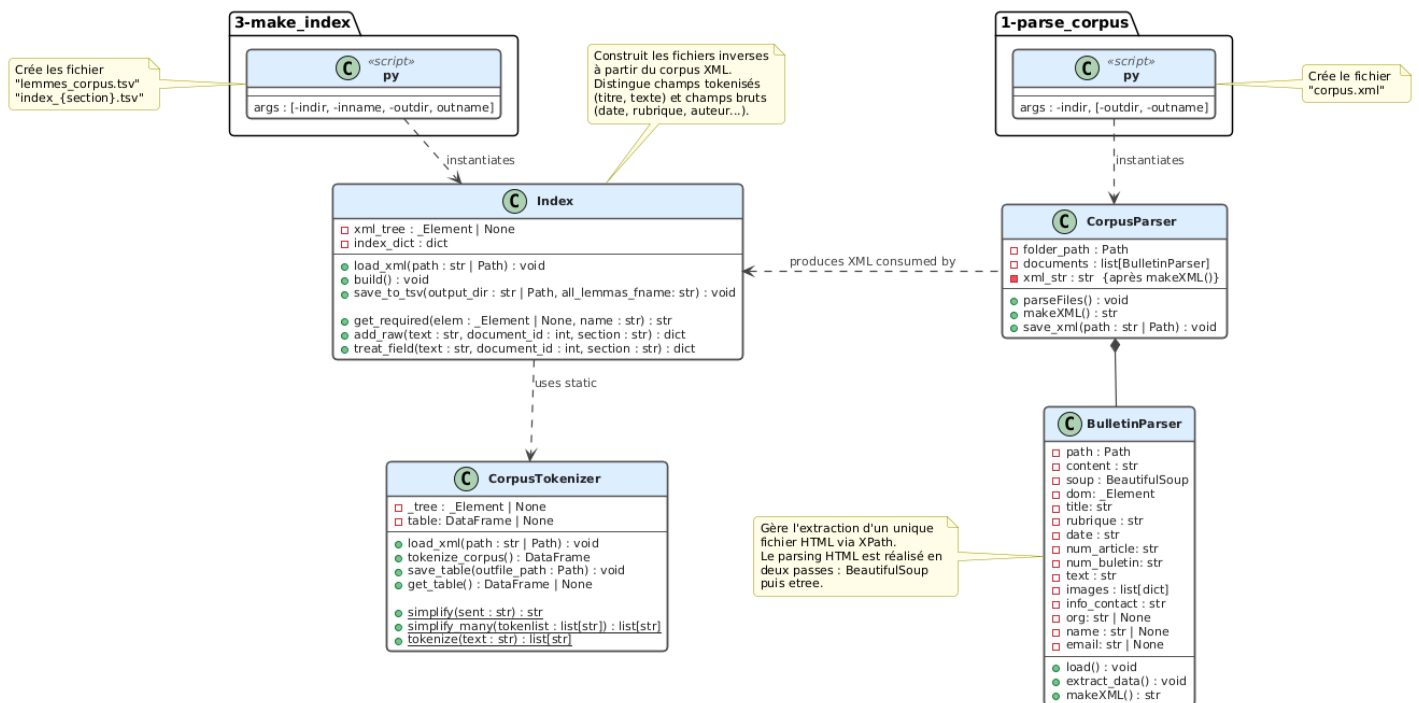


FIGURE 1.1 – UML Indexation (TD1)



Partie 2

Anti-dictionnaire

L'objectif ici est de construire un anti-dictionnaire (une liste de mots non porteurs de sens à exclure du corpus), en s'appuyant sur le coefficient TF-IDF pour les identifier automatiquement.

2.1 Choix de l'unité documentaire

On retient l'**article** comme unité documentaire plutôt que le bulletin, car un bulletin regroupe des rubriques hétérogènes qui dilueraient le signal TF-IDF. L'article, centré sur un seul sujet, rend la fréquence des tokens plus informative et cohérente avec l'unité utilisée pour la recherche.

2.2 Calcul du TF-IDF

Le calcul est réalisé par la classe `TFIDFProcessor`. Le TF est la fréquence brute du token dans le document, la fréquence normalisée n'étant pas nécessaire ici puisqu'on cherche à identifier les tokens les plus fréquents globalement. L'IDF est calculé selon : $idf_t = \log_{10}(N/df_t)$

où N est le nombre de documents. Avant de calculer df_t , le nombre de documents contenant t , on déduplique les couples (`document_id`, `token`) pour ne compter qu'une seule fois un token apparaissant plusieurs fois dans un même document. On n'applique pas de lissage : un token présent dans tous les documents obtient un TF-IDF nul (car son IDF est de $\log(1) = 0$).

Le TF-IDF final est le produit $tf_{t,d} \times idf_t$. Les tableaux contenant ces trois métriques pour chaque document du corpus sont sauvegardés dans des fichiers TSV séparés.

2.3 Détermination de l'anti-dictionnaire

La classe `AntiDict` identifie les stopwords en calculant le TF-IDF **moyen** de chaque token sur l'ensemble des documents où il apparaît. On retient la moyenne plutôt que la médiane ou le maximum : elle reflète le niveau global de discrimination des tokens et permet de prendre en compte les faiblesses récurrentes, contrairement au maximum qui dépend d'un seul pic, et à la médiane qui peut masquer certaines variations.

Le seuil de $tf-idf \leq 0.7$ a été retenue pour discriminer les stop-words des mots à garder. Cette valeur a été trouvée à partir de données, en les analysant avec le notebook `antidictionnaire.ipynb`.

Tout token dont la moyenne est inférieure ou égale au seuil est ajouté à l'anti-dictionnaire, stocké dans un set Python pour des recherches en $O(1)$ lors de l'exécution.

La méthode `build_sub_table()` de `AntiDict` permet de l'avoir sous forme d'une table de substitution à deux colonnes (`token`, `sub`), où une chaîne vide "" indique la suppression. Cette méthode est utile pour la classe `DataCleaner` qui se base sur ce format.

2.4 Génération du corpus sans stop word

Le corpus sans stop word est déterminé avec le script `td2-determination_antidict.py`, en particulier par la fonction `substitue()` qui applique la table de substitution de l'anti-dictionnaire au corpus. Ce script réponds à l'énoncé du TD, et permet d'avoir le fichier `corpus_filtre.xml` nécessaire pour les prochains TD. Cependant, il n'est pas utilisé dans le programme finale, la logique ayant ensuite été déplacée dans la classe `DataCleaner` qui effectue toute la pipeline de nettoyage de données sans fichiers intermédiaires.



Partie 3

Lemmatisation et indexation du corpus

L'objectif de ce TD est de normaliser le vocabulaire par lemmatisation, d'affiner l'anti-dictionnaire à la lumière de cette normalisation, et de construire les fichiers inverses définitifs.

3.1 Lemmatisation du corpus filtré

3.1.1 Classes Stemmer

On définit une classe abstraite `Stemmer` exposant une interface commune (`make_table`, `transform`, `transform_tolist`, `save_table`), dont héritent `SpacyStemmer` et `SnowStemmer`. Cette conception les rend interchangeables dans le pipeline sans modifier le code appelant.

3.1.2 Implémentation des deux stemmers

`SpacyStemmer` s'appuie sur le modèle français `fr_core_news_sm`. Plutôt que de soumettre le texte brut à `spaCy`, on lui fournit directement les tokens produits par `CorpusTokenizer` en construisant manuellement un objet `spacy.tokens.Doc`. Cela garantit une tokenisation cohérente avec le reste du pipeline et évite tout désalignement avec l'index.

`SnowStemmer` utilise le `SnowballStemmer` de NLTK configuré pour le français. Chaque token est raciné indépendamment via `model.stem()`, sans contexte grammatical : l'approche est déterministe et rapide, mais aveugle aux nuances morphologiques.

3.1.3 Analyse comparative et choix

Le script `exercices-td/td3-make_stemmer_tabs.py` applique les deux stemmers sur `nostopwords_corpus.x` et sauvegarde les tableaux `token/lemma` pour analyse avec le notebook `analyse_stem.ipynb`. Sur notre corpus de **14 344 tokens uniques**, `SpaCy` produit **11 285 lemmes uniques** et `Snowball` **9 046 racines uniques**. La courbe de Pareto confirme que les deux approches suivent un profil similaire, avec environ 20% des lemmes couvrant 80% du corpus, `Snowball` ayant toutefois une distribution plus extrême. Ce qui est logique, car il réduit plus agressivement le vocabulaire.

La différence qualitative est plus décisive : `Snowball` réduit *optimisation* et *optimiste* à la même racine `optim`, causant des collisions sémantiques problématiques pour un moteur de recherche. **On retient `SpaCy`** : on perd en rappel (l'utilisateur doit être plus précis), mais on gagne en précision en évitant ces confusions.

3.2 Affinage de l'anti-dictionnaire

La lemmatisation regroupe plusieurs formes fléchies sous un même lemme, augmentant mécaniquement la fréquence de certains termes jusque-là sous le seuil. Par exemple, *permettre* avait un TF-IDF moyen de 0.92 avant lemmatisation ; après, l'absorption de *permet*, *permettent*, *permis* et *permettait* fait chuter cette moyenne à 0.42. De même, *son* absorbe *sa*, *leur* et *leurs*. En maintenant le même seuil $\text{tfidf} \leq 0.7$, on élimine ces nouveaux termes sans ajuster les hyperparamètres. Le corpus doublement filtré est produit par `2-clean_data.py`.



3.3 Construction des fichiers inverses

La classe Index est appliquée au corpus doublement filtré et lemmatisé. On obtient un fichier TSV par champ indexé : `index_titre.tsv`, `index_texte.tsv`, `index_rubrique.tsv`, `index_date.tsv`, `index_auteur.tsv`, `index_contact.tsv` et `index_image.tsv`, chacun contenant le lemme, sa fréquence totale et la liste des identifiants d'articles associés.

Les index sont donc conservés sur le disque sous forme de fichiers TSV, puis convertis en dictionnaires Python lors de leur utilisation. Les avantages de cette approche sont qu'elle permet de visualiser facilement les index inversés et, après le chargement en mémoire RAM, la recherche s'effectue en $O(1)$, car le dictionnaire Python implémente une **table de hachage** (*hash map*).

Cependant, cette méthode suppose de pouvoir charger l'ensemble du dictionnaire en mémoire vive. Dans notre cas, le corpus est petit, donc cela ne pose pas de problème. En revanche, avec un corpus plus volumineux, ou susceptible de croître fortement, il devient plus pertinent d'utiliser des **B-trees**, qui constituent le standard pour l'indexation sur disque. Ils permettent de minimiser les accès au disque dur tout en évitant de surcharger la mémoire RAM.

3.4 La classe DataCleaner

L'ensemble du pipeline de nettoyage du corpus (anti-dictionnaire et lemmatisation) est réalisé par la classe `DataCleaner`. Cette classe se sert de la classe `AntiDict` pour déterminer les stop words et d'un `Stemmer` (`SpacyStemmer` par défaut), pour lemmatiser le texte dans le texte et le titre (les autres champs sont laissés tels quels). `DataCleaner` génère un tableau de substitution avec des paires token/lemme, mais où lemme peut être vide si c'est un stop word.

Pour exporter la liste définitive des stop words choisis pour le corpus (automatiques + manuelles), la méthode `export_stop_words()` sauvegarde les stop words dans un fichier texte contenant simplement les stop words.

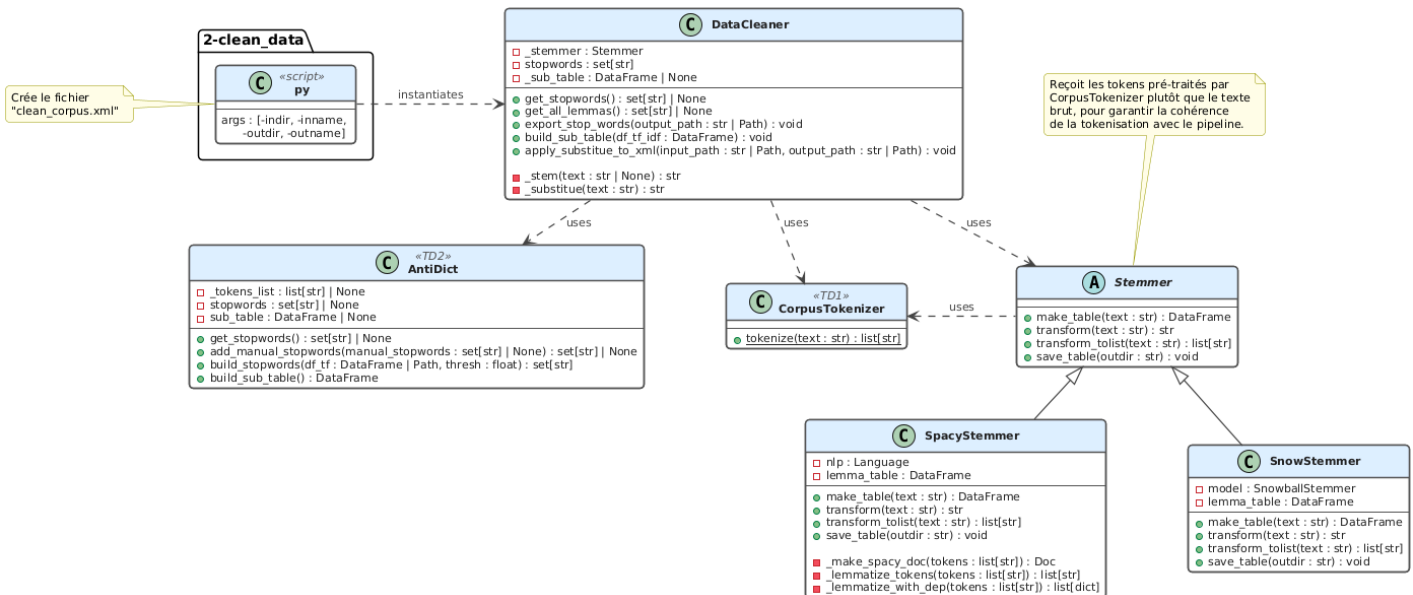


FIGURE 3.1 – UML Stemmer + AntiDict (TD2 et TD3)



Partie 4

Correcteur orthographique

L'objectif de ce TD est de construire un correcteur orthographique capable, pour chaque token absent du lexique, de proposer le lemme le plus proche. Ce module est intégré dans la classe `Pretraiteur`, qui constitue également le point d'entrée du traitement des requêtes (TD5).

4.1 Pipeline de correction

Le traitement de chaque token suit cinq étapes séquentielles dans `extract_key_words` : tokenisation et lemmatisation via `SpacyStemmer`, filtrage des stop words par comparaison à l'anti-dictionnaire, détection des entités numériques via conversion en float, vérification dans le lexique, et enfin correction orthographique pour les tokens absents.

Le lexique est maintenu sous deux formes : `self.lemmas` (liste ordonnée) pour l'itération dans la recherche par préfixe, et `self.lemma_set` (ensemble) pour les tests d'appartenance en $O(1)$.

4.2 Génération des candidats par recherche par préfixe

La méthode `generate_candidates` réduit l'espace de recherche avant d'appliquer Levenshtein. Elle s'appuie sur trois hyperparamètres fixés empiriquement : `seuilMin = 3` (mots de moins de 3 caractères ignorés des deux côtés), `seuilMax = 4` (différence de longueur maximale tolérée) et `seuilProx = 0.6` (ratio de caractères identiques en position sur la longueur maximale). Un arrêt anticipé est déclenché dès que le taux d'erreur accumulé dépasse $1 - \text{seuilProx}$, ce qui évite de parcourir des mots manifestement trop différents.

4.3 Départage par distance de Levenshtein

`treat_non_existant` applique la logique suivante selon la taille de la liste de candidats : si elle est vide, le token est abandonné ; si elle contient un seul candidat, il est retourné directement ; si elle en contient plusieurs, on retourne celui qui minimise la distance de Levenshtein. Celle-ci est implémentée par programmation dynamique en $O(m \times n)$, avec un coût de substitution binaire. L'approche en deux temps - préfixe puis Levenshtein - limite le calcul quadratique à un petit sous-ensemble de candidats.

4.4 Vulnérabilité de l'approche et alternative

La recherche par préfixe compare les caractères position par position depuis le début du mot. Si la faute porte sur le **début du mot**, le score de proximité sera nul et le bon candidat sera écarté avant même que Levenshtein ne soit appelé. Par exemple, *erecehrche* n'aura aucun caractère commun en position avec *recherche*. Pire encore, avec l'algorithme proposé dans le cours, la recherche s'arrête dès la première différence, donc même oublier un accent en début de mot peut causer un score de 0. Notre solution permet au moins de résoudre le deuxième cas.

Une alternative robuste consiste à utiliser des **n-grammes de caractères** : on représente chaque mot par ses bigrammes ou trigrammes, puis on calcule une similarité de Jaccard entre les ensembles. Cette méthode est insensible à la position de l'erreur et tolère des transpositions ou insertions en n'importe quel endroit du mot.



Partie 5

Traitement des requêtes

L'objectif de cette partie est de construire un pipeline qui transforme une requête en langue naturelle, en un format utilisable par notre moteur de recherche, c'est-à-dire une recherche booléenne. Notre solution extrait les champs étape par étape (date, images, rubriques, etc.), en enlevant les parties extraites de la requête avant de la transmettre à l'étape suivante.

5.1 Format d'une requête

Nous représentons les opérations logiques sous *forme normale disjonctive* (DNF) : les listes internes correspondent aux conjonctions (*ET*), tandis que la liste externe représente la disjonction (*OU*) entre ces conjonctions. Ainsi, « A ou B et C » est représentée par `[[A], [B,C]]`.

Contenu d'une requête formatée :

- `type_doc` : le type de document recherché (article ou rubrique entière), article par défaut.
- `date_from`, `date_to` : les bornes de la date du document (None s'il n'y a pas de borne).
- `anti_date` : les dates que l'on veut éviter (ex : "pas au mois de juin" → `*/06/*`).
- `image` : booléen indiquant s'il faut une image ou pas (None si cela n'a pas d'importance).
- `rubrique` : liste indiquant les rubriques souhaitées.
- `contenu` : contient le DNF pour les mots clés portant sur le contenu.
- `titre` : contient le DNF pour les mots clés portant sur le titre.
- `exclude` : contient la liste conjonctive des termes à exclure de la requête.

5.2 Extraction des dates

La classe `DateExtractor` extrait les dates d'une requête, qu'elles soient numériques ou en langage naturel. Elle utilise deux types de regex : celles détectant une date dans son contexte et celles la convertissant au format `jj/mm/aaaa`. La méthode `extract()` identifie intervalles, dates min/max, dates exactes ou anti-dates, puis transmet les fragments à `_parse_date_fragment()` pour conversion.

5.3 Extraction des mots clés

Une des plus grandes difficultés a été l'extraction des mots clés, car il y a beaucoup de variabilité dans les requêtes. La classe en charge de cette partie est `KeywordExtractor`. Pour cette partie, il a fallu déterminer une liste de stop words propres aux requêtes, car par exemple "je" et "veux" ne sont peut-être pas des stop words du corpus, mais dans les requêtes ils ne sont pas importants.

Pour extraire les mots clé, il faut d'abord extraire les parties de la requête les contenant. `RE_TITRE` et `RE_CONTENU` permettent de distinguer les mots clés du titre et du contenu, il y a aussi d'autres regex pour des cas particuliers. Une fois ces bouts de texte extraits, ils sont traités par la méthode `_parse_logic_block()` qui construit le DNF pour les mots clés. Elle détecte les opérateurs de conjonction via des regex et sépare le texte logiquement. Si le mot n'est pas un stop word ni du corpus ni des requêtes, alors il est lemmatisé et corrigé par `normalize_terms()`.



5.4 Classe Pretraiteur

La classe Pretraiteur encapsule tout le pipeline pour passer d'une requête brute à une requête formatée. Elle utilise les classes DateExtractor et KeyWordExtractor pour extraire les dates et les mots clés. Pour les traitements plus simples (bulletin, image ou non, etc.), la logique est directement mise dans des méthodes de Pretraiteur. C'est la méthode `treat_request()` qui effectue la totalité du pipeline. Chaque traitement retire la partie traitée, ce qui permet d'éviter les conflits.

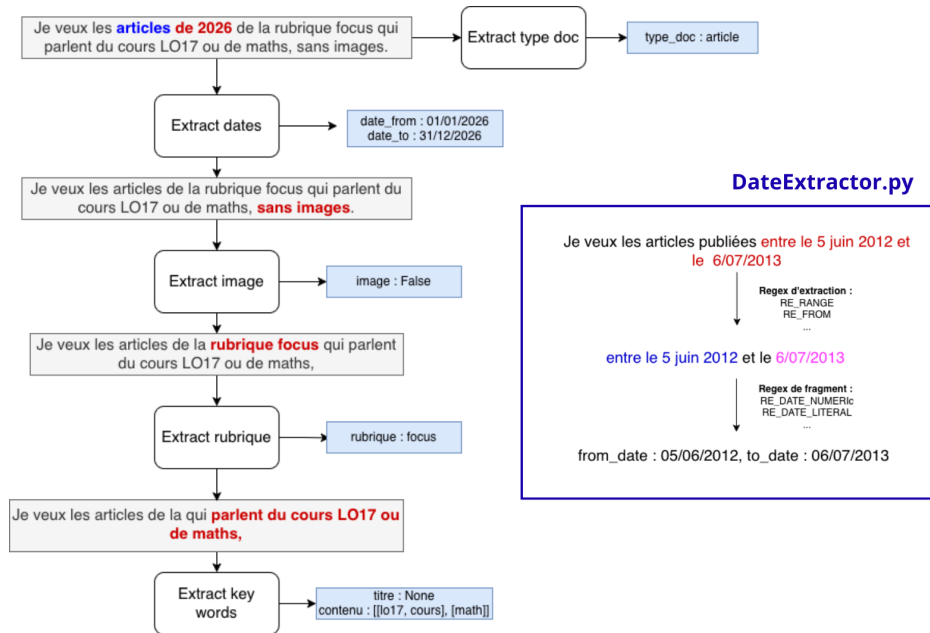


FIGURE 5.1 – Pipeline de traitement des requêtes (Pretraiteur + DateExtractor)

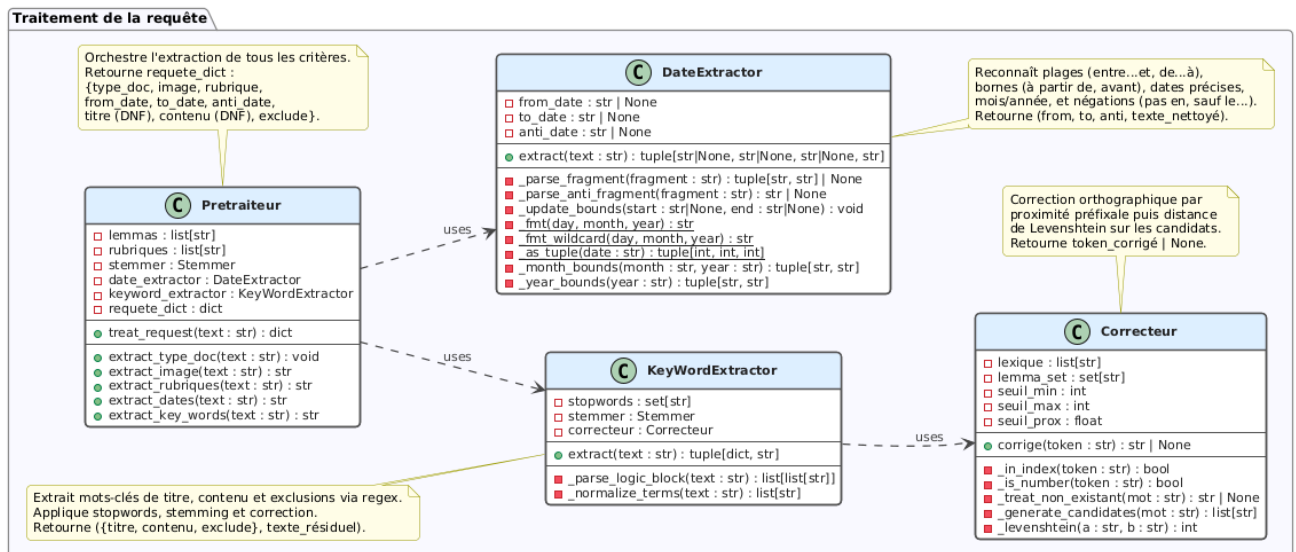


FIGURE 5.2 – UML Pretraiteur + Correcteur (TD4 et TD5)



Partie 6

Moteur de recherche et évaluation

Une fois que nous avons des fichiers d'index inversés, et une pipeline qui permet de transformer des requêtes naturelles en requêtes booléennes, il suffit de construire le moteur de recherche qui trouve les documents pertinents à partir de la requête.

6.1 Chargement de l'index

Comme spécifié en partie 3.3 de ce rapport, nous avons choisi de sauvegarder les index inversées sous format TSV pour être convertis en dictionnaire Python lors du chargement en RAM. Le chargement est fait par `SearchEngine` lors de son initialisation. La structure résultante est un dictionnaire qui associe les tokens à des sets Python contenant les ids des documents associés. La structure set de Python implémente les opérations de union et intersection dont nous avons besoin, mais nous devons les compléter dans les méthodes `_union` et `_intersect` simplement pour prendre en compte des cas limites.

6.2 Recherche

La méthode `search(requete_dict)` de `SearchEngine` permet d'effectuer la recherche de documents respectant les contraintes en utilisant les opérateurs d'union et intersection.

6.3 Classement des résultats

Notre recherche est booléenne, donc les documents correspondent ou non : il ne s'agit pas d'une recherche vectorielle. Cependant, nous pouvons quand même classer les résultats en nous basant sur différents poids selon les champs. Par exemple, même si deux documents contiennent le terme recherché, celui qui l'a à la fois dans son titre et dans son contenu doit passer avant celui qui ne l'a que dans son contenu. Ainsi, nous avons modifié la formule du cours pour définir :

$$\text{score_doc} = \frac{1}{n_{mc}} \sum_{i=0}^l p_i \sum_{j=0}^{n_{mc}} \delta(i, j)$$

Où n_{mc} est le nombre de mots clés dans la requête, l est le nombre de champs (titre, contenu), p_i est le poids du i -ème champ, et $\delta(i, j) = 1$ si le j -ème mot clé appartient au i -ème document.

6.4 Interface en ligne de commande et web

Nous avons développé une interface pour utiliser le moteur de recherche en ligne de commande située dans `scripts/4-moteur.py`.

Pour avoir une meilleure expérience utilisateur, nous avons également réalisé une interface web en utilisant Flask. La programmation web sortant du cadre de l'UV LO17, l'IA générative a été utilisée pour guider la réalisation du template `templates/index.html` et de l'API `app.py` selon nos besoins. L'application web contient donc 4 interfaces : le moteur de recherche utilisant `SearchEngine`, le catalogue de l'entière du corpus, une interface pour annoter manuellement le corpus (avec annotation automatique des contraintes de dates), une interface pour évaluer le moteur de recherche sur l'ensemble des requêtes manuellement annotées enregistrées.

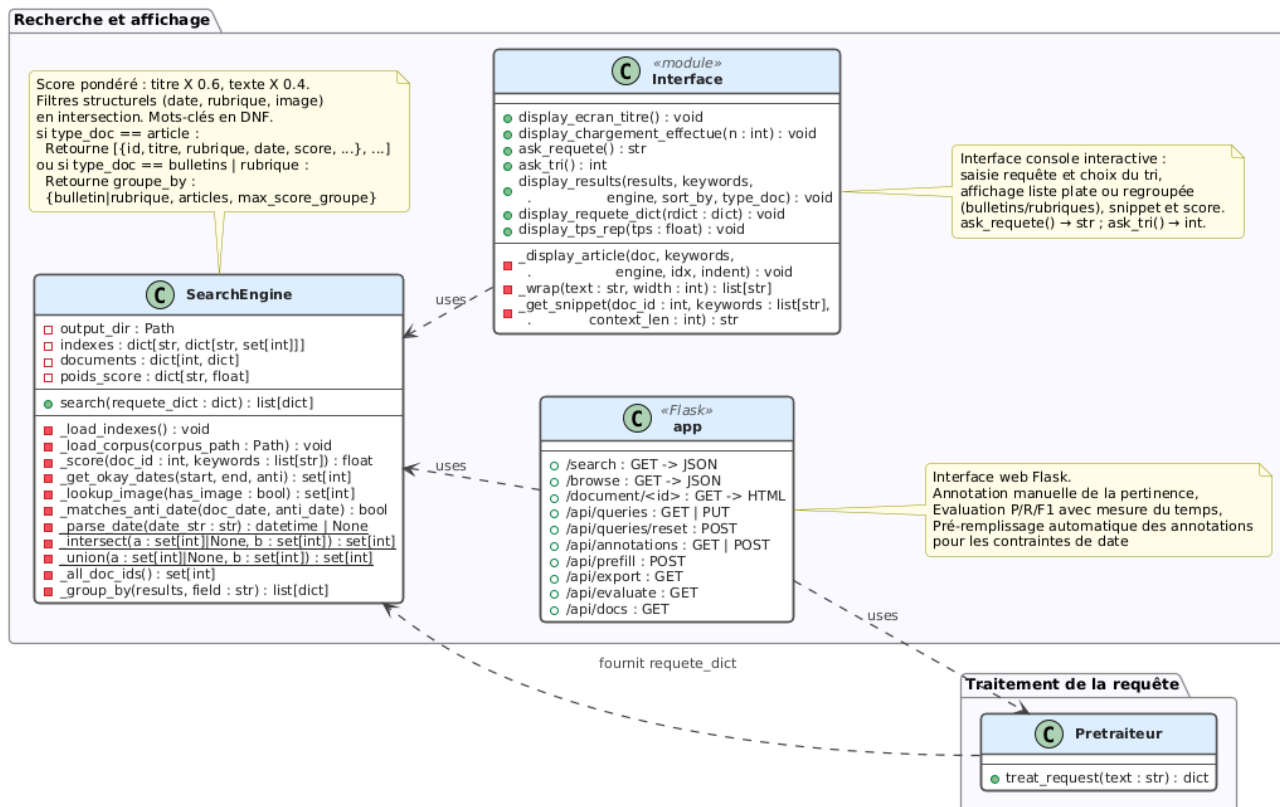


FIGURE 6.1 – UML Moteur de Recherche + Interface (TD6)

6.5 Évaluation du moteur de recherche

Afin de connaître les performances de notre moteur de recherche, nous pouvons l'évaluer avec des métriques de précision et de rappel. Pour calculer ces métriques, il faut connaître les documents qui sont pertinents et non pertinents pour certaines requêtes, et donc il est nécessaire de les annoter manuellement. Nous avons donc annoté l'ensemble du corpus pour 10 requêtes suffisamment complexes pour bien tester le modèle.

Les requêtes utilisées sont les suivantes :

0. Quels sont les articles parus entre le 3 mars 2013 et le 4 mai 2013 évoquant les États-Unis.
1. J'aimerais la liste des articles écrits après janvier 2014 et qui parlent d'informatique ou de télécommunications.
2. Je veux les articles qui parlent des systèmes embarqués et non pas de la robotique.
3. Je veux voir les articles de la rubrique Focus et publiés entre le 30/08/2011 et le 29/09/2011.
4. Quels sont les articles traitant d'informatique ou de réseaux ?
5. Chercher les articles dans le domaine industriel et datés à partir de 2012.
6. Je veux les articles de 2014, de la rubrique Focus et parlant de la santé.
7. Quels sont les articles dont le titre contient le terme "marché" et le mot "projet" ?
8. Je veux les articles qui parlent du fauteuil roulant et qui ont pour rubrique Actualité Innovation.
9. Listez-moi les articles qui parlent de 3D et qui sont écrits entre 2010 et 2011.

L'annotation manuelle a été faite de façon à annoter seulement les documents réellement pertinents, et non seulement ceux qui contiennent les mots clés. Par exemple, un document qui



ne mentionne pas l'informatique mais parle de "logiciel" sera quand même pertinent pour une requête traitant de l'informatique. À l'inverse, un document qui contient les mots "système" et "embarqué" mais qui ne parle pas vraiment de systèmes embarqués ne sera pas pertinent.

Notre système atteint sur ces requêtes une **précision moyenne de 0.83**, un **rappel moyen de 0.72** et un **F1 moyen de 0.79** (figure 6.2). On remarque une meilleure précision que rappel ; cela vient du fait que souvent les documents contenant les mots clés sont pertinents, mais que beaucoup de documents sans ces mots clés le sont aussi car ils utilisent d'autres termes.

Une des requêtes (la numéro 7) demandait que le titre contienne *le terme "marché" et le mot "projet"*. Or le lemmatiseur SpaCy comprend mal le contexte et transforme "marché" en "marcher". Aucun document n'a le terme marcher et projet dans le titre, donc la requête ne retourne rien alors qu'elle devrait. Cela met en évidence une difficulté liée au fait que nous utilisons un lemmatiseur, qui doit se baser sur le contexte, plutôt qu'un stemmer qui a une approche plus basique.

Certaines requêtes mettent en évidence les limites de l'approche *bag of words* que nous employons. Par exemple, la requête 4 demande *traitant d'informatique ou de réseaux*. On comprend que le terme réseau ici fait référence aux réseaux informatiques, mais des documents portant sur des réseaux d'entreprise, des réseaux cellulaires ou encore des réseaux électriques sont retournés, faisant baisser la précision. Pour ces cas, il faudrait prendre en compte plus finement l'ordre des mots et la sémantique du texte. De même pour la requête 5 traitant du *domaine industriel*. Un document qui mentionne le mot "industriel" n'est pas forcément pertinent pour la requête, pourtant il correspond tout à fait aux contraintes booléennes.

Le temps des requêtes est aussi un facteur important (figure 6.3). En exécutant chacune des 10 requêtes 100 fois, nous avons une bonne estimation du temps moyen. Celui-ci est de 0.012 ms. La requête 7 est particulièrement longue, prenant en moyenne environ 6 fois plus de temps que les autres. Cela est dû à la correction automatique : puisque le terme "marcher" doit être corrigé pour devenir "marche", il passe par la correction. Or "marcher" est un mot de longueur très commune, donc le préfiltre laisse passer beaucoup de mots (3640 exactement), alors qu'un mot comme "supramoléculaire" par exemple est très long et n'a que très peu de candidats d'une taille similaire (909 exactement).

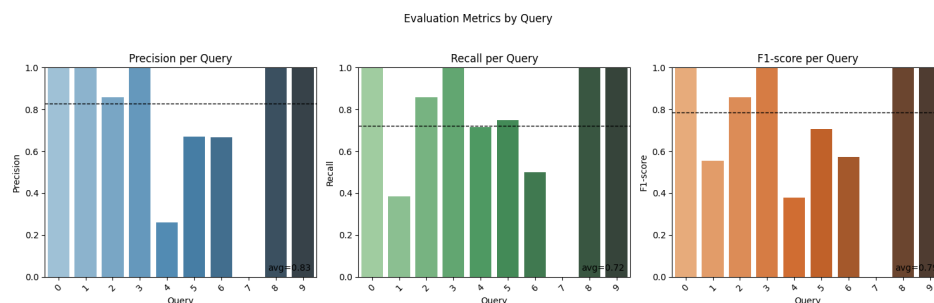


FIGURE 6.2 – Métriques d'évaluation de la pertinence du moteur de recherche

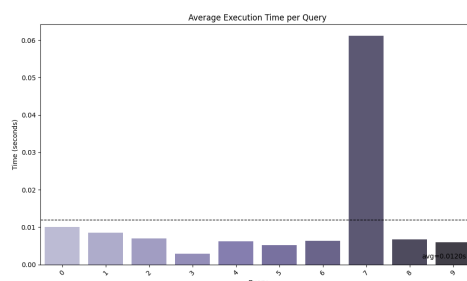


FIGURE 6.3 – Métriques d'évaluation du temps d'exécution du moteur de recherche



Conclusion

Résumé du travail

Dans ce travail, nous avons construit un moteur de recherche capable de traiter des requêtes en langage naturel et d'effectuer des recherches booléennes sur le corpus de l'ADIT.

Ce DM a permis de mettre en application plusieurs notions du cours sur un cas concret, tout en mettant en évidence les difficultés liées au traitement automatique du langage naturel.

Le projet a impliqué le scraping des documents, la détermination de l'anti-dictionnaire à l'aide d'une analyse fondée sur le TF-IDF, la lemmatisation du corpus, la construction d'index inversés, le traitement des requêtes en langage naturel à l'aide d'expressions régulières, ainsi que l'appariement entre les requêtes et les documents pertinents via le moteur de recherche.

Une fois le moteur finalisé, il a été évalué sur 10 requêtes pour lesquelles les documents pertinents ont été annotés manuellement. Cette évaluation a mis en évidence les limites de la recherche booléenne ainsi que celles de l'approche *bag of words*.

Une progression naturelle consisterait à s'éloigner de la recherche booléenne en implémentant une recherche vectorielle simple utilisant le TF-IDF. En allant plus loin, il serait également possible de s'éloigner de l'approche *bag of words*, et d'utiliser des modèles de Deep Learning afin d'obtenir des représentations vectorielles denses, plus riches sémantiquement, et tenant compte du contexte.

Bilan

En résumé, ce travail inclut :

- le scraping de pages web ;
- la construction d'index inversés avec anti-dictionnaire et stop words ;
- le traitement de requêtes en langage naturel ;
- la construction d'un moteur de recherche booléen avec l'approche *bag of words* ;
- l'évaluation du moteur de recherche sur un échantillon à l'aide des métriques de précision, rappel et score F1.

Contributions

Les responsabilités sur ce projet se sont beaucoup chevauchées, avec une grande part de réflexion menée en collaboration. Au total, la division du travail est restée équilibrée, chaque membre ayant apporté un niveau de contribution comparable au projet.

Ci-dessous le détail des contributions spécifiques :

Caio ARTUS (contributions : 50%) : implémentation du scraping des pages HTML (Parser), détermination de l'anti-dictionnaire, implémentation du tokenizer, implémentation des classes Stemmer, implémentation du constructeur d'index, implémentation de la pipeline de prétraitement (DateExtractor, KeywordExtractor, Pretraiteur), contributions à la classe SearchEngine, contributions à l'interface web et console, réflexions autour du classement des résultats, analyses diverses sur les notebooks pour la justification des choix et l'évaluation finale du moteur de recherche, rédaction de parties du rapport.

Clothilde MARKO-LAFON (contributions : 50%) : contribution à l'implémentation du tokenizer, implémentation de la TF-IDF et du DataCleaner reprenant l'anti-dictionnaire et les classes Stemmer, implémentation du correcteur par Levenshtein, implémentation du moteur de recherche à partir d'un dictionnaire de requête, développement de l'interface console et web, plus généralement refactorisation et correction du code selon une architecture POO afin de mettre en évidence les fonctionnalités clés implémentées à chaque TD, création de script clairs et propres (exercices-td/ et scripts/), rédaction et mise en page du rapport, création des UML.